M1 : IOT (Internet of Things)

Few words on JSON . . .

Gilles Menez

Université Côte d'Azur
email : menez@unice.fr
www : www.i3s.unice.fr/~menez

March 24, 2026: V 1.0

Table of Content

So many good existing courses . . .

The objective is NOT to make another full course on JSON !

You will find many good things on the WEB :

- https://www.w3resource.com/JSON/introduction.php
- https://la-cascade.io/json-pour-les-debutants/
- . . .

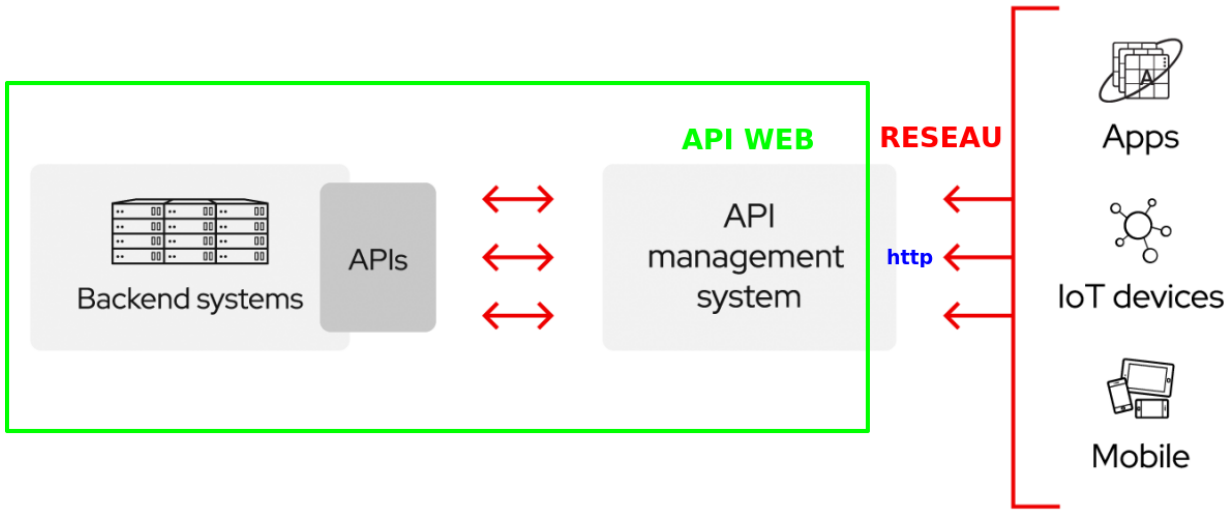
But, we could spend few slides on JSON API for languages/system as C/Python/. . .

and have a look at JSON **in the ESP/NodeRed context** used in your future homeworks.

When Apps communicates . . .

Applications are creating and processing with data structures.

- As they telecommunicate too, a recurrent issue is to exchange such structures . . . through API for example :



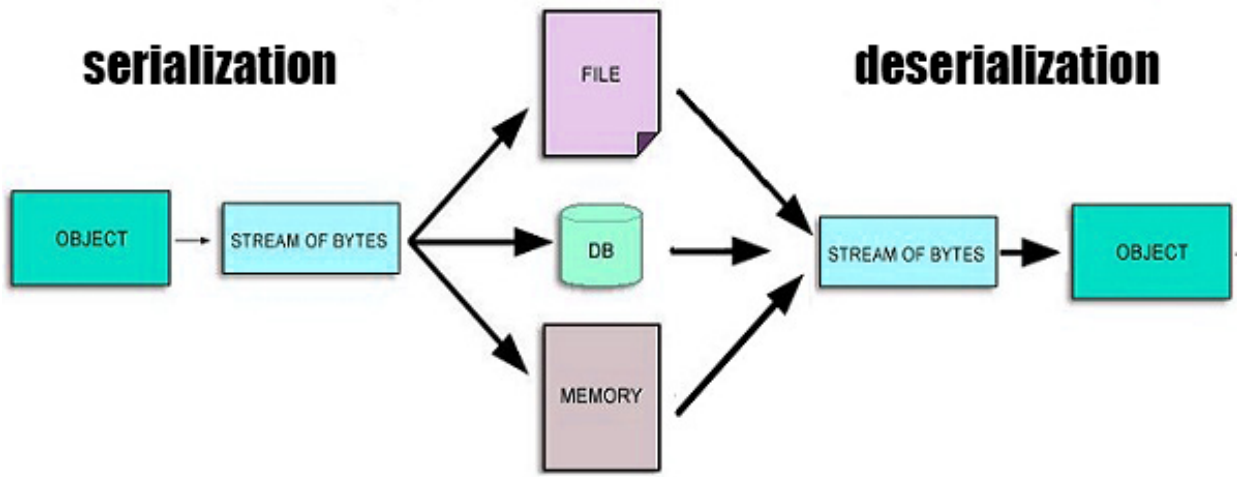
Direct manipulations of the digital information (as dumps of memories of these structures ?) will be **highly non-interoperable**, . . .

=> **"serialization" was proposed** !

Serialization

Serialization is a technique

- that **translates structured data (in memory ?) into a format (a string ?)** and
- that **permits recovery of its original structure** (Deserialization).



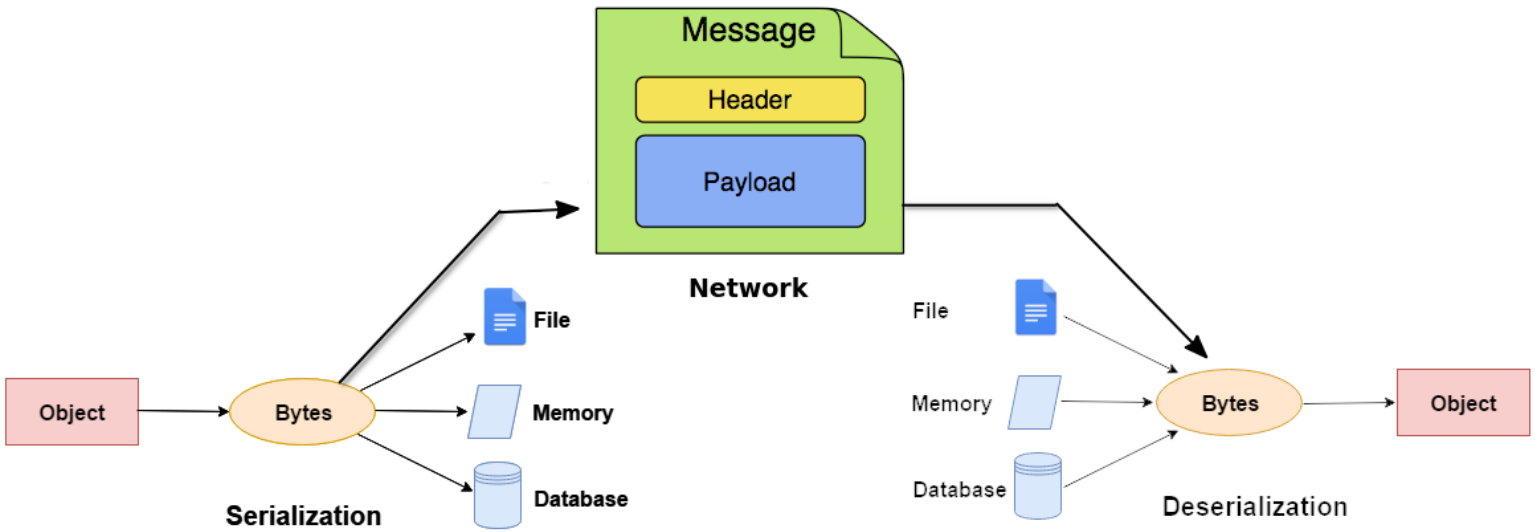
https://en.wikipedia.org/wiki/Serialization

It **enables sharing** of the digital information through storage or transmission.

Payload and which serialization ?

Good ! . . . So, **payload of exchanged packets will be serialized** !

- Which syntax ?
- Which semantic ?
- . . .



A **standardized format** (preferable to custom implementations due to lower time investment and improved interoperability) that can be easily shared across various applications.

Most known "format" candidates

Choosing this format should depend on factors such as human readability, the complexity of data, and storage, speed limitations, . . .

- https://blog.mbedded.ninja/programming/serialization-formats/a-comparison-of-serialization-formats/
- https://en.wikipedia.org/wiki/Comparison_of_data-serialization_formats

1. JavaScript Object Notation (JSON)

   JSON is a **Human-readable** language that one can easily understand what it says.

   It has a Map structure and it is a **"fast" data serialization language**.

Most known "format" candidates (cont.)

2. Extensible Markup Language (XML)

   XML has originally developed for dynamic web pages but its properties allow to make it a data serialization language.

   It has a tree structure.

XML vs. JSON

```xml
<person>
  <name>John Smith</name>
  <age>25</age>
  <address>
    <street>21 2nd Street</street>
    <city>New York</city>
    <state>NY</state>
    <postalCode>10021</postalCode>
  </address>
  <sex>
    <type>male</type>
  </sex>
</person>
```

```json
{
  "name": "John Smith",
  "age": 25,
  "address": {
    "street": "21 2nd Street",
    "city": "New York",
    "state": "NY",
    "postal code": "10021"
  },
  "sex": {"type": "male"}
}
```

Most known "format" candidates (cont.)

3. YAML Ain't Markup Language (YAML)
   It is a light-weight, human-readable data-representation language.

   It is primarily designed to make the format easy to read while including complex features.

Since YAML is **a superset of JSON**, it can parse JSON with a YAML parser.

Most known "format" candidates (cont.)

You can have comments in Yaml . . . not in Json :-(

https://www.snaplogic.com/fr/blog/json-vs-yaml-whats-the-difference-and-which-one-is-right-for-your-enterprise

4. Protocol Buffer : (by Google)

   https://developers.google.com/protocol-buffers/docs/overview

5. MessagePack

   https://msgpack.org/

6. eXternal Data Representation (XDR) :

   https://en.wikipedia.org/wiki/External_Data_Representation

7. . . .

Most known "format" candidates (cont.)

JSON is not necessary the best solution :

see more !

but often use and as all others often compare themself to JSON :-)

. . . let's use JSON !

Example of Payload in JSON

Imagine your IoT application object wants to send the following information to another peer on the network :

"Now, the temperature is 21 degrees celsius"

Instead of a natural language string, please consider the JSON format example below :

```
{
  "SensorType" : "Temperature",
  "Value" : 21,
  "Hist" : {"Cnt" : 3, "Last" : [20,21,23]},
  "Tol"  : true
}
```

The syntax only is the standard ...

```
{
  "SensorType" : "Temperature",
  "Value" : 21,
  "Hist" : {"Cnt" : 3, "Last" : [20,21,23]},
  "Tol"  : true
}
```

This message contains more than an instantaneous value (cf "Last" Key) and is quite "wordy" ... why not ?

Anyway is there a standard ?

NOOOOOOOO !!!! and this a HUGE problem for interoperability :-(

JSON : JavaScript Object Notation

Converting a native object to a string (so it can be transmitted across the network) is called **serialization**.

- From this moment, **JSON is a text-based data format** and as a consequence **JSON exists as a string** (. . . to be a payload !)

app obj − **Serialization** − > "string" − **Deserialization** − > app obj

It needs **to be converted to** a native language **object** when you want to access the data structure serialized in your program.

- **Converting a string to a native object** (so it can be manipulate by app) is called **deserialization**.

JavaScript/Python/Java/. . . all provide a global JSON object that has methods available for converting between the two.

JSON : JavaScript Object Notation (cont.)

You can find quite the same basic data types inside JSON objects as there exist in a JavaScript/Java/Python/... object :

1. **four basic and built-in data types** : strings, numbers, booleans and null
2. **two structured data types** : arrays, and other object literals.
   - **Objects** are a list of **label-value** pairs.

     Objects are wrapped within { and }.
   - **Arrays** are list of values.

     Arrays are enclosed by [and].

   Both objects and arrays can be nested.

This allows you to construct a data hierarchy, like the example that follows ...

In this example the top level JSON object contains a key that identifies the ESP :

```
1   { "ESP ident" : "deathstar",
2     "sensors" : [
3     {
4       "Type" : "Temperature",
5       "Model" : "dhb11",
6       "Value" : 21,
7       "Hist" : {"Cnt" : 3, "Last" : [20,21,23]},
8       "Tol"  : true
9     },
10    {
11      "Type" : "Light",
12      "Model" : "photoresistor",
13      "Value" : 987
14    }
15    ],
16    "DD" : { "lat" : 43.62 , "lng" : 7.05}
17  }
```

- This object encloses (with label "sensors") an array of sensors . . . available on the EVM / used by ESP ? Each sensor is itself an object, . . .
- The value associated to the "DD" label is a JSON object dedicated to the geolocalisation of the ESP.

The poor approach !

Write your JSON without "help" . . . and bother with syntax !

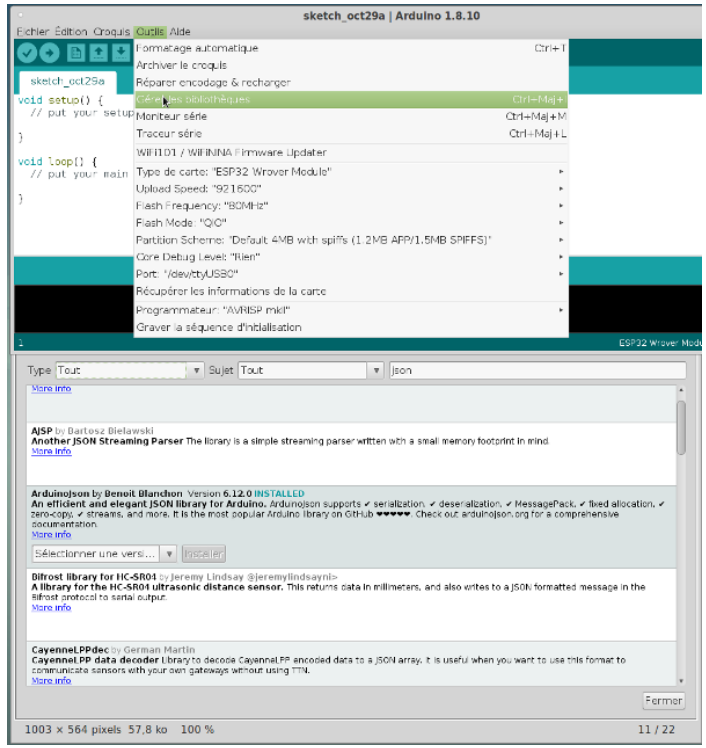
```
1   String jsonData = "{\"machine_id\":";
2   jsonData += "\"M0001\"";
3   jsonData += ",\"date\":";
4   jsonData += "\"2019-July-06\"";
5   jsonData += ",\"time\":";
6   jsonData += "\"10:15:30\"";
7   jsonData += ",\"machine_power_state\":";
8   jsonData += "\"1\"";
9   jsonData += ",\"machine_operation_state\":";
10  jsonData += "\"1\"";
11  jsonData += ",\"powerkW\":";
12  jsonData += "\"12.34\"";
13  jsonData += ",\"model_being_tested\":";
14  jsonData += "\"abcd_123\"";
15  jsonData += ",\"number_tested_today\":";
16  jsonData += "\"1234\"";
17  jsonData += ",\"number_passed_today\":";
18  jsonData += "\"1234\"";
19  jsonData += "}";
```

If you try to rewrite this . . . probably you will loose at least one char or a field.

- A good API will propose a better / more confortable solution to write this !

“ArduinoJson” by B.Blanchon for ESP32

- https://arduinojson.org
- https://github.com/bblanchon/ArduinoJson



It can be easily installed in Arduino IDE, through the library menu : ”Outils/Gérer les bibiliothèques”.

Examples

You will find numerous examples and use cases :

- https://arduinojson.org/v6/example/
- https://techtutorialsx.com/2017/04/27/esp32-creating-json-message/

You will find advices too :

- https://techtutorialsx.com/2019/05/02/esp32-arduinojson-v6-serializing-json/

Remember IoT has the same constraints as desktop machines for example on memory print of objects :

- "Use **String** objects sparingly, because ArduinoJson duplicates them in the JsonObject.
- **Prefer plain old char[]**, as they are more efficient in term of code size, speed, and memory usage".

A tiny test of the ESP32 JSON API

First of all, **why should I use an API** ?

- I think the major advantage is to manipulate JSON object and leave serialisation to API methods
- I don't want to build the string by myself with sprintf neither analyze the received string by myself !

Let's see an exemple : Imagine my ESP have a GPS sensor and wants to send its location in a JSON format :

```
{
  "sensor":"gps",
  "time":1351824120,
  "data":[48.756080,2.302038]
}
```

```
1   /*
2    * Simulation of exchanges of GPS data in JSON format
3    * Fichier : appp_json/appp_json.ino
4    * Modifie par G.MENEZ
5    */
6
7   #include <ArduinoJson.h>
8   void setup() {
9     Serial.begin(9600);
10    Serial.println();
11  }
12  void loop() {
13    char payload[256]; // THE payload : string exchanged between ESP and ???
14
15    /* 1) Build the JSON object ... easily with API !*/
16    StaticJsonDocument<256> jdoc; // Why static ?
17                        //=> https://arduinojson.org/v6/api/staticjsondocument/
18    jdoc["sensor"] = "gps";
19    jdoc["time"]   = 1351824120;
20    JsonArray data = jdoc.createNestedArray("data"); // "data" field is an array
21    data.add(48.756080);
22    data.add(2.302038);
23
24    /* 2) SERIALIZATION  => fill the payload string from jdoc object */
25    //serializeJson(jdoc, Serial); // This one serializes DIRECTLY in Serial => console
26    // so prints on {"sensor":"gps","time":1351824120,"data":[48.756080,2.302038]}
27    serializeJson(jdoc, payload); // This one put the serialized Json Obj in a STRING
28    //payload<="{\"sensor\":\"gps\",\"time\":1351824120,\"data\":[48.756080,2.302038]}";
29
```

```
30      /* 3) Send the request to the network  and Receive the answer */
31      Serial.println("Emission of the String/Payload ");
32      Serial.println(payload);
33
34      /* 4) DESERIALIZATION => build a new JSON object from the payload */
35       StaticJsonDocument<256> jdoc_new;
36       deserializeJson(jdoc_new, payload);
37       Serial.println();
38       Serial.println("Reception and Deserialization of the String/Payload ");
39
40       /* 5) Uses of this new object */
41       const char* sensor = jdoc_new["sensor"];
42       long time          = jdoc_new["time"];
43       double latitude    = jdoc_new["data"][0]; // an array in the "data" field !
44       double longitude   = jdoc_new["data"][1];
45       Serial.print("Sensor : "); Serial.println(sensor);
46       Serial.print("Time : "); Serial.println(time);
47       Serial.print("Latitude : "); Serial.println(latitude);
48       Serial.print("Longitude : "); Serial.println(longitude);
49       Serial.println("\n==============================");
50
51       // Elapsed time between loops
52       delay(10000);
53   }
```

API : "Server Side"

On the ESP, in C, you build the JSON object and ask for its serialization. All modern languages (Java, Python, Javascript ...) offer a JSON API that will directly work on native objects.

When converting a Python object in JSON, following matchings are operated :

Python	JSON
dict	Object
list	Array
tuple	Array
str	String
int	Number
float	Number
True	true
False	false
None	null

cf https://docs.python.org/fr/3/library/json.html

In Python :

```
1   import json
2
3   # Received Payload : some JSON formatted string
4   recv_payload =  '{"sensor":"gps", \
5                     "time":1351824120,\
6                     "data":[48.756080,2.302038]}'
7   print(recv_payload)
8
9   #== DESERIALISATION  : From JSON string to Python dict
10  # Parse and Work With ...
11  d = json.loads(recv_payload)
12  # the result is a Python dictionary :
13  print(d["data"])
14  d["time"] = 0
15
16  #== SERIALISATION : Convert Python dict to JSON string
17  sent_payload = json.dumps(d) # the result is a JSON string
18  # ... READY TO BE SENT !!
19  # or to be printed
20  nice = json.dumps(d, indent=2, sort_keys=True, separators=(", ", ": "))
21  print(nice)
```

In this Teaching Unit, as we make "simple" things, we could use a simpler API : **simplejason**

https://stackoverflow.com/questions/712791/what-are-the-differences-between-json-and-simplejson-python-modules

simpler ! . . . and faster ?

Cyphering the payload ? : AES-128-CBC

```
1  from cryptography.fernet import Fernet
2
3  #--- Encryption engine
4  #Generation dynamique d'une cle
5  cipher_key = Fernet.generate_key()
6  print(cipher_key)
7
8  # On peut aussi a prtir de l'exemple en creer une et la partager.
9  cipher_key=b'p7lqIMC18kWn64cZFGe9lZvl4iv4UQeG41miCFLwRQc='
10 cipher = Fernet(cipher_key)
11
12 #----Original
13 payload = b'on'
14 print("\nOriginal message =", payload)
15 #----Encryption
16 encrypted_payload = cipher.encrypt(payload)
17 print("\nEncrypted message =", encrypted_payload)
18 #------------ Transmission -----------
19 #----Decryption
20 decrypted_payload = cipher.decrypt(encrypted_payload)
21 print("\nreceived message =", str(decrypted_payload.decode("utf-8")))
22
```

In the ESP, you will find cyphering library . . . BUT **what about electrical consumption** ?

TODO : A Json model of the ESP

In this section, as a TODO, I "should" ask you to "propose a Json model of the ESP32 with temperature, light, . . . sensors".

- Like a "status" we could send to a monitoring station . . .

BUT if I do and give you to much freedom, we will obtain as many Json structures as there are students in the room : **AND THAT IS A/THE MAJOR ISSUE of IOT** !

In a near future, when we will build a fleet of objects we will not be able to understand each others !

So you will be given (here after) a "pre defined JSON model (by GM)" of the ESP32.

* For creating the String of the Json object, I USED the pretty print function ("`serializeJsonPretty()`") of the Json document (cf ArduinoJSON)

```
{
  "status": {
    "temperature": 25.9375,
    "light": 0123,
    "regul": "HALT",
    "fire": true,
    "heat": "OFF",
    "cold": "OFF",
    "fanspeed": 1234
  },
  "location": {
    "room": "312",
    "gps": {
      "lat": 43.62453842,
      "lon": 7.050628185
    },
    "address": "Les lucioles"
  },
  "regul": {
    "lt": 26,
    "ht": 25.89999962
  },
  "info": {
    "ident": "ESP32 123",
    "user": "GM"
    "loc": "A Biot"
  },
  "net": {
    "uptime": "55",
    "ssid": "Livebox-B870",
    "mac": "AC:67:B2:37:C9:48",
    "ip": "192.168.1.45"
  },
  "reporthost": {
    "target_ip": "127.0.0.1",
    "target_port": 1880,
    "sp": 2
  }
}
```

Few comments :

Some fields have a semantic that will be presented/used in the next courses.

- In this case, you just put a constant or a "NOP".

Other fields,

- in "status" section, should be easy to fullfill from sensors values.
- "regul" section are the thresholds used for regulation
- "info" and "location" are constants you can choose by yourself.

```
1   #ifndef MAKEJSONH
2   #define MAKEJSONH
3   /*
4     Proposition of a (arduino side) model for the ESP
5     File : status_json/makejson.h
6    */
7   #include <ArduinoJson.h>
8   typedef struct {
9     int luminosity;
10    float temperature;
11
12    const float highThreshold = 26.0;
13    const float lowThreshold = 25.9;
14    bool coolerState;  // "ON" : "OFF";
15    bool heaterState; // "ON" : "OFF";
16    bool regulationState; // Regulation status : "RUNNING" or "HALT"
17    bool fireDetected;
18    int fanSpeed;
19
20    // Résidence Newton, 2400 Route des Dolines, 06560 Valbonne, France
21    const double latitude = 43.62454;
22    const double longitude = 7.050628;
23    char room[50] ="312";
24    char address[200] = "Les lucioles";
25    // Network
26    String WiFiSSID;
27    String MAC;
28    String IP;
29    // Reporting
30    String target_ip = "127.0.0.1";
31    int target_port = 1880;
32    int target_sp = 2;
33  } esp_model;
34
35  /* Make a JSON Doc from location GPS */
36  StaticJsonDocument<1000> makeJSON_fromlocation(float lat, float lgn);
37  /* Make a JSON Doc of the esp status */
38  StaticJsonDocument<1500> makeJSON_fromstatus(esp_model *em);
39  #endif
```

JSON Validation

Vous allez émettre et recevoir des schémas JSON et au moins deux questions se posent immédiatement :

1. Est ce que "ce" JSON respecte la syntaxe JSON ?
2. Est ce que "ce" JSON respecte le schéma convenu par l'app ?

Il existe des outils, sous différentes formes, pour répondre à ces questions.

- Par exemple, l'outil en ligne https://jsononline.net/fr/json-validator permet de savoir si "ce" JSON est syntaxiquement valide.

Au niveau de la programmation, la conséquence logique est de chercher à valider le contenu de "ce" JSON en le confrontant à un "modèle".

- Là encore il existe des validateurs dans tous les langages de programmation qui permettent d'exprimer un "JSON schema document" et d'utiliser ce modèle pour valider "ce" JSON.

  https://json-schema.org/implementations

JSON Schema

While JSON is probably the most popular format for exchanging data, JSON Schema is the vocabulary that enables JSON data consistency, validity, and interoperability at scale.

To be sure we talk of the good/same concepts as our peer, we could have a look at :

https://json-schema.org/

The following url is a good introduction to JSON format concept/terminology/...

https://json-schema.org/learn/getting-started-step-by-step

And this one shows that W3C Web works on providing standardized building blocks that make use of JSON Schema :

https://json-schema.org/blog/posts/w3c-wot-case-study

TODO : JSON Validation

Comme je sais comment on va se servir de ces JSON dans le dernier devoir, j'ai choisi pour vous deux outils qui implémentent "JSON Schema":

1. En Python, la bibliothèque "jsonschema" :

   https://python-jsonschema.readthedocs.io/en/stable/

   https://builtin.com/software-engineering-perspectives/python-json-schema

2. En nodeJS, la bibliothèque "Ajv" (Another JSON Schema Validator) :

   https://ajv.js.org/

   https://simonplend.com/get-started-with-validation-in-node-js/

3. Je suis ouvert à d'autres outils . . .

TODO : JSON Validation

En utilisant une de ces deux bibliothèques, **concevoir un validateur de la donnée JSON que vous produisez à partir de l'ESP.**

Pour l'instant,

1. vous placez votre json dans un fichier,
2. et votre validateur doit dire si "ce" JSON est valide conformément au schéma dont je vous ai donné une instance possible.

   Je vais le tester avec des Json délibérément " non conformes".

   - Blindez votre validateur !

Vous pouvez écrire votre validateur en Python ou en JS.

Plus tard (TP5) je vais vous demander un serveur en Python ou en JS et que ce serveur intégrera ce validateur pour valider le JSON qu'il reçoit par le réseau.

JSON "standardisation"

Cela ne va pas être simple de se mettre d'accord ...

https://dzone.com/articles/json-http-and-the-future-of-iot-protocols